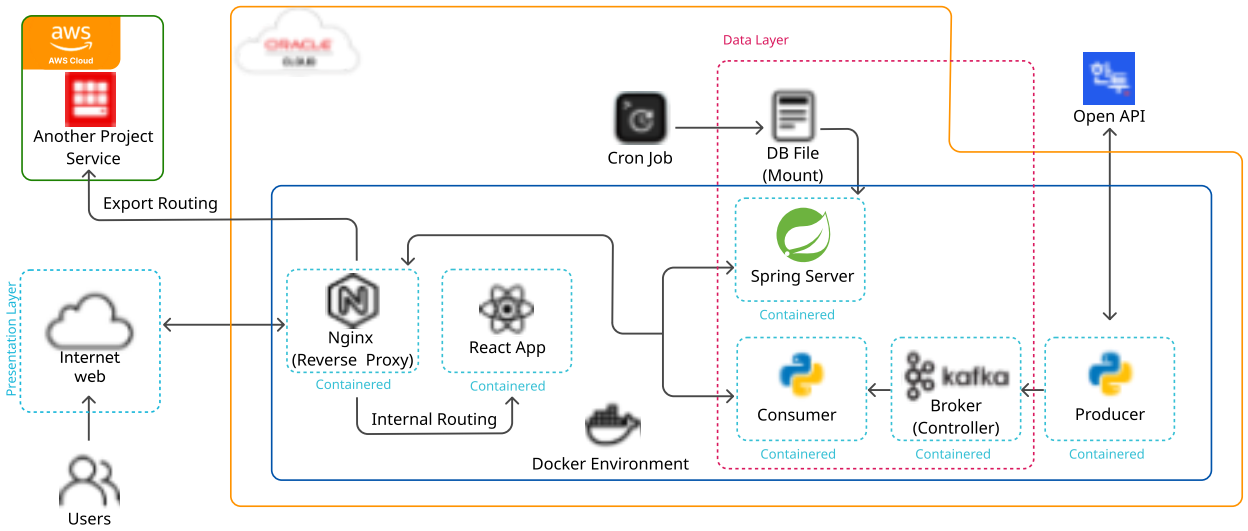
실시간 차트 데이터 파이프라인 구성

- 기존 프로젝트에서 담당했던 실시간 차트 기능을 구현 및 배포

Architecture



Presentation & Routing

- HTTPS 기반 단일 진입점(Single Entry Point) 구축:
  - Nginx를 활용해 모든 외부 요청을 443 포트로 단일화하여 인프라 복잡성을 은닉하고 보안을 강화했습니다
  - HTTP(80) 요청에 대한 자동 HTTPS(443) 리다이렉션 설정을 적용하여, 모든 사용자 데이터 통신이 암호화된 채널을 통해서만 이루어지도록 강제했습니다.
- 경로 기반 라우팅:
  - 요청 경로에 따라 정적 자원, API요청, 웹소켓 연결을 각각의 전용 컨테이너로 분기하는 구조
- External Proxy 확장성:
  - 향후 인스턴스 시스템 부하를 고려하여, Nginx 설정 변경만으로 다른 프로젝트 서비스에 트래픽 넘기는 방식 가능

Application Layer

- 어플리케이션 컨테이너화:
  - 프론트엔드, 백엔드, 데이터 수집, 데이터 전파를 각각 독립된 컨테이너로 분리하여 마이크로서비스 지향적 구조 설계
- Internal DNS:
  - IP를 통한 내부 통신 대신 도커 내부 DNS를 활용해 통신함으로써, 컨테이너 재시작 시에도 끊김 없는 연결성 확보
- OutBound 통신:
  - 외부 API와 통신하는 데이터 수집 서비스는 프록시 거치지 않고 직접 연결하여 네트워크 지연 최소화

Live Streaming Pipeline

- 이벤트 기반 아키텍처 Kafka 도입:
  - Kafka를 중앙 브로커로 두어 데이터 생산과 전파를 관리하고, 대량의 메세지 스트림을 중계

- Fan-out 기반 데이터 전파:
    - Consumer 컨테이너 내부에 구현된 웹소켓 서버가 다수의 클라이언트들에게 데이터를 전파하도록, 인메모리 기반의 팬아웃 구조 적용하여 서버 자원 효율성 확보
- 비동기 데이터 파이프라인 최적화:
    - 데이터 수집을 다중화하고, 전파 흐름을 비동기적으로 처리하여, 병목 현상 최소화

Data & Infra

- Docker Bridge Network:
    - 인스턴스의 내부 통신을 도커의 가상 망으로 격리하여, 외부에서 내부 서비스로 직접 접근하는 것 방지
- Volume Mount를 통한 영속성 확보:
    - 호스트 시스템의 db파일을 컨테이너에 매핑하여, Stateless 컨테이너 환경에서도 데이터 상태 보존

서비스 컨테이너화 현황

```
ubuntu@instance-20250916-1337:~$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                   STATUS          PORTS
broker                  Up 16 minutes   9092/tcp
stock-front-server      Up 3 days       80/tcp
stock_consumer          Up 8 days
producer_1              Up 8 days
producer_2              Up 8 days
stock-backend-server    Up 2 weeks
nginx-with-encrypt      Up 8 days       0.0.0.0:80->80/tcp, :::80->80/tcp, 0.0.0.0:443->443/tcp, :::443->443/t
ubuntu@instance-20250916-1337:~$
```

📌 고민

💡 문제점(Challenge)

- 증권사 Open API(WebSocket)를 통해 수집되는 실시간 체결 데이터는 변동성이 커질 경우 급증하기 때문에 **단일 파이프라인으로 구조로는 특정 체결 데이터에의해 병목이 발생하게 되면** 나머지 종목 체결 데이터가 줄줄이 밀리는 연쇄 지연 발생

🛠️ 해결 방안(Solution)

- 데이터 수집 단의 병렬화
    - Producer 역할을 수행하는 프로그램을 별도 컨테이너에서 구동해 **수집 속도 개선**
- 토픽 기반 데이터 샤딩
    - 2개의 토픽으로 티커들은 분리하여 단일 토픽에 모든 트래픽이 몰릴 경우 발생하는 브로커의 **부하를 분산시킴**
    - 추가적으로 균등한 부하를 목적으로 Grafana, Prometheus를 활용한 kafa-monitoring 시스템을 활용하여 각각의 토픽별로 **균등한 트래픽을 유지하는지 모니터링 기능 추가 시킴



- 메시지 키 기반 순서 보장
    - 토픽 내에서 티커를 Key로 구분하여 체결 종목 데이터의 **인과 관계를 보장**

✅ 결과 및 성과



- 데이터 전달 과정에서의 지연 최소화로 동시 업데이트 체감

💡 문제점(Challenge)

- 초기 컨슈머 설계 단계에서는 클라이언트가 접속할 때 마다 해당 요청을 처리하기 위해 독립적인 컨슈머를 **매번 새로 생성하여 할당**하는 방식이었고, 접속자 수가 늘어날수록 브로커와의 커넥션이 선형으로 증가하기에 서버 자원 소모가 극심해 질 수 있었고, 동일한 종목을 조회하는 여러 클라이언트가 있어도 각 컨슈머가 브로커로부터 동일한 데이터를 중복해서 수신하는 네트워크 낭비도 발생했음

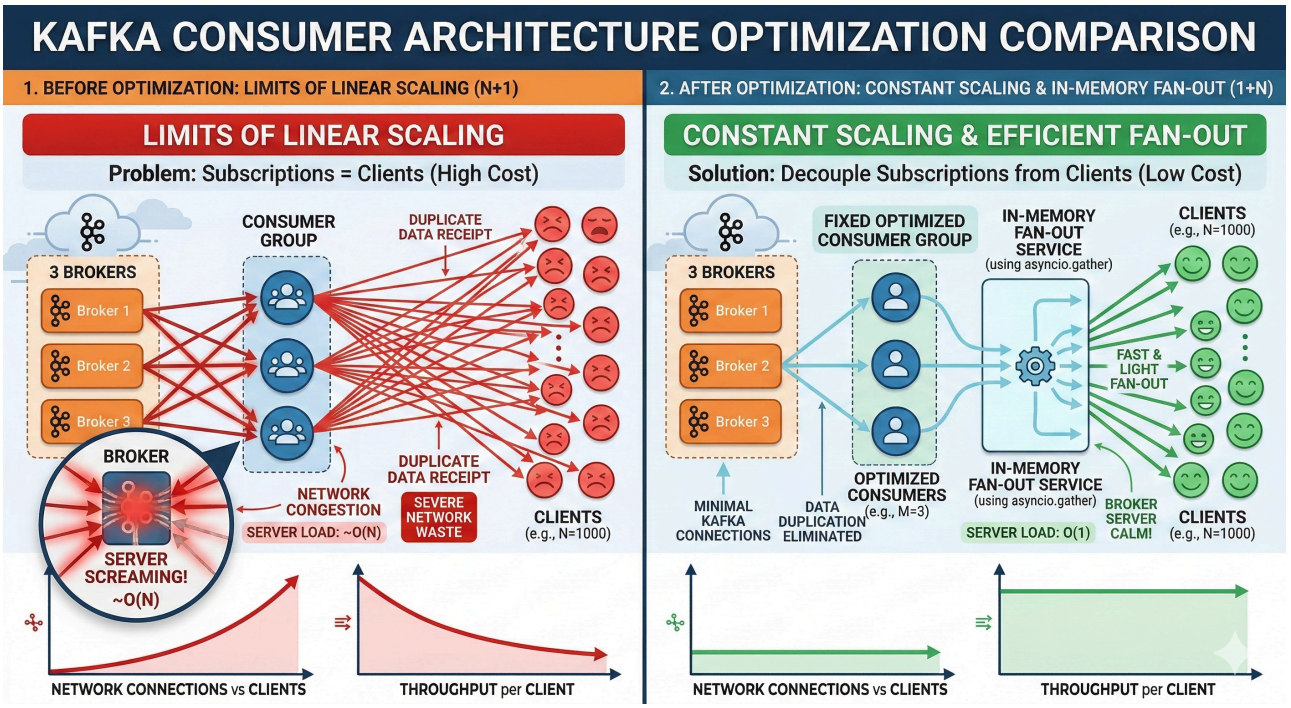
🛠️ 해결 방안(Solution)

- 컨슈머 계층의 독립:
    - 클라이언트의 접속 여부와 상관 없이, 시스템에서 필요한 데이터를 topic, ticker별로 고정된 수의 컨슈머만 운용하도록 설계 변경

- 비동기 태스크 관리:
    - asyncio.gather을 이용해서 클라이언트들에게 BroadCast하는 방식으로 로직을 병렬화 수행, 이를 통해 특정 클라이언트의 네트워크 지연이나 예외가 전체 파이프라인에 병목을 일으키지 않도록 격리
- 인메모리 팬아웃(Fan-out) 구조 구현:
    - 컨슈머가 브로커로부터 데이터를 딱 한 번 읽어오면, 이를 서버 내 전역 상태에서 관리하는 모든 클라이언트들에게 비동기로 전파하는 구조 구현

✅ 결과 및 성과

- 해결 방안 적용 전, 후 비교·분석



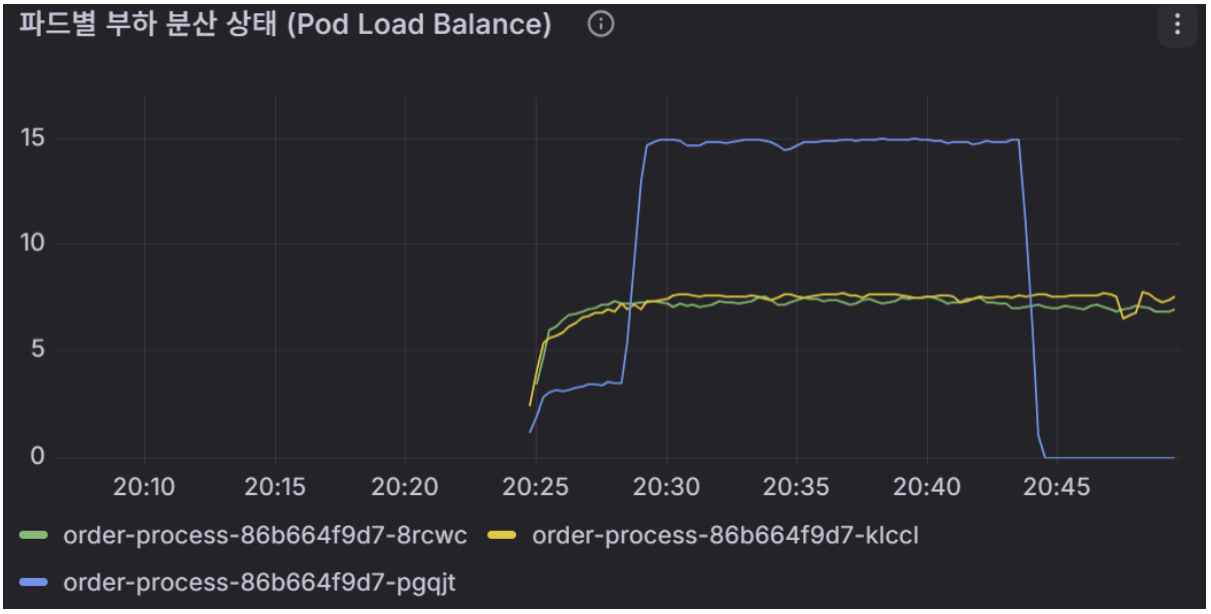
📌 k3s 인프라 전환 및 매수 시스템 구현 및 트랜잭션 고도화 (k6 부하 검증)

- 기존 단일 Docker 환경을 k3s(경량 쿠버네티스) 오케스트레이션 인프라로 전환하고, 동시 매수 트랜잭션 시 발생한 DB 병목을 개선하여 k6 부하 테스트(500 VUs)로 성능을 검증한 과정

📌 k3s 오케스트레이션 인프라 구축

- Replica 배포 및 로드밸런싱:
    - k3s 내장 Load Balancer와 Replica 배포 전략을 연계하여 다중 Pod로 트래픽을 균등 분산

    - Pod의 부하 분산 상태를 통해 실제 주문 프로세스의 처리의 흐름을 파악



- 장애 복구(Failover) 검증:
    - 특정 인스턴스/포드 장애 발생 시 인프라가 이를 감지하고 가용 노드에 Pod를 자동 재생성하는 오케스트레이션 프로세스 검증 및 가용성 확보

💡 문제점(Challenge)

- 동시 매수 주문이 폭주할 경우, 데이터 일관성 및 주문 순서를 보장하기 위해 적용했던 DB 비관적 락(Pessimistic Lock)으로 인해 락 경합(Lock Contention) 및 쓰기 병목 발생
- 가상 사용자(500 VUs) 급증 시 DB 커넥션 풀 대기 시간이 늘어나 전체 시스템 성능이 저하되고 응답 지연이 심화되는 한계 직면

🛠️ 해결 방안(Solution)

- DB Lock 제거 및 이벤트 드라이븐 아키텍처 전환
    - DB의 비관적 락을 철회하고 Kafka 기반 비동기 매수 주문 파이프라인으로 구조 개선
- user_id 메시지 키 기반 파티셔닝
    - Kafka 메시지 발행 시 user_id를 Partition Key로 지정하여 동일 유저의 주문 순서(Ordering)는 보장하면서, 서로 다른 유저의 트래픽은 복수 파티션으로 균등 분산 처리
- k6 활용 500 VUs 부하 테스트 진행
    - 동일한 자원 및 동시 접속자 환경(500 VUs)에서 DB 비관적 락 적용 방식과 user_id 파티셔닝 방식의 성능 데이터를 정량적으로 비교 측정

✅ 결과 및 성과

- k6 부하 테스트(500 VUs) 성능 비교 분석

측정 지표 (Metric)	기존 (DB 비관적 락)	개선 (user_id 파티셔닝)	개선 결과
HTTP TPS (처리량)	83.32 req/s	158.81 req/s	약 90.6% 향상
총 성공 주문 건수	22,444 건	42,697 건	약 90.2% 증가
평균 응답 시간 (avg)	3.68 s	1.91 s	약 48.1% 단축

측정 지표 (Metric)	기존 (DB 비관적 락)	개선 (user_id 파티셔닝)	개선 결과
p(90) 응답 시간	5.86 s	2.90 s	약 50.5% 단축
p(95) 응답 시간	6.65 s	3.42 s	약 48.6% 단축

- DB Lock을 제거하고 user_id 파티션 키 기반 비동기 구조로 전환하여 **TPS를 약 2배 향상**시키고, 응답 지연 시간을 **전 구간 절반 수준으로 단축**

```
TOTAL RESULTS

  checks_total.......: 42821   156.975434/s
  checks_succeeded...: 100.00% 42821 out of 42821
  checks_failed......: 0.00%   0 out of 42821

  ✓ Valid Response (2xx or 400/422 Insufficient Balance)
  ✓ SSE Subscribed or Maintained Stream

  CUSTOM
  successful_orders..............: 42697  156.520868/s

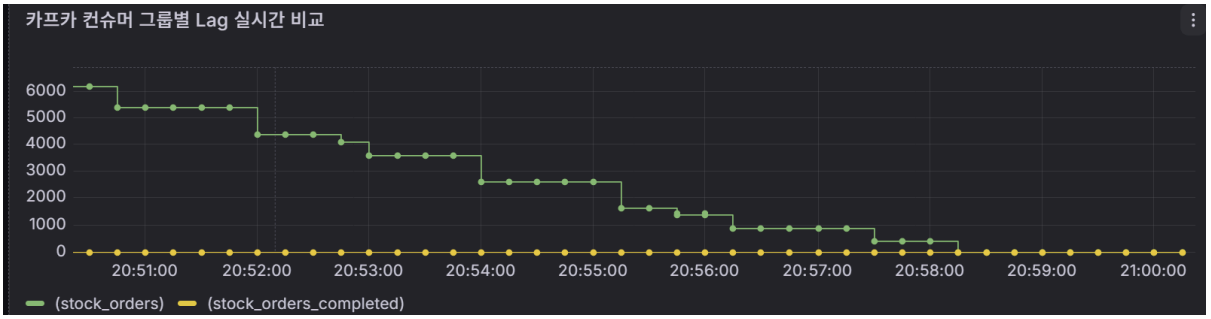
  HTTP
  http_req_duration..............: avg=1.91s min=3.2ms   med=1.5s  max=2m0s  p(90)=2.9s  p(95)=3.42s
    { expected_response:true }...: avg=1.57s min=3.2ms   med=1.5s  max=7.99s p(90)=2.88s p(95)=3.38s
  http_req_failed................: 0.28%  124 out of 43321
  http_reqs......................: 43321  158.808359/s

  EXECUTION
  iteration_duration.............: avg=2.03s min=103.6ms med=1.62s max=2m1s  p(90)=3.01s p(95)=3.53s
  iterations.....................: 42821  156.975434/s
  vus............................: 1      min=0           max=500
  vus_max........................: 500    min=500         max=500

  NETWORK
  data_received..................: 18 MB  65 kB/s
  data_sent......................: 9.5 MB 35 kB/s
```

🔍 한계 분석 및 향후 고도화 방향 (Next Steps)

- **Consumer LAG 및 DB Write 병목 관찰**
  - HTTP 요청 응답 및 처리량(TPS) 측면에서는 큰 성능 개선을 이루었으나, 주문 건별 직접 db에 쓰기 작업으로 인한 Consumer LAG 및 DB I/O 부하의 구조적 한계를 관찰함



- **차기 아키텍처 개선 로드맵**
  - **Redis 기반 In-Memory 연산 및 Bulk Write 도입:** 유저 잔액 및 주문 상태 데이터를 Redis 인메모리로 격리하여 연산 속도를 극대화하고, 처리된 성공 트랜잭션을 비동기 Bulk Write 방식으로 DB에 일괄 저장하여 Disk I/O 병목 해소 추진
  - **Kafka Partition 확장 및 Consumer Pod 스케일 아웃:** stock_orders 토픽의 파티션 개수를 늘리고, k3s 오케스트레이션을 활용해 Pod를 스케일 아웃하여 메시지 소진 속도 향상 도모

💡 문제점(Challenge)

- **메인 API 쓰레드의 Kafka ACK 대기 병목**
  - 매수 주문 요청 처리 시 Kafka Broker의 ACK 응답을 동기로 대기할 필요 없음에도, 기존 HTTP 요청 Tomcat Thread가 이를 대기하며 동시 처리량 저하 발생
- **주문 완료 notification을 사용자에게 전달하기 위한 SSE(Server Sent Event) 기능 추가 후**
  - 주문 완료 알림을 전달하기 위한 SSE 지속 연결 작업에 톰캣 쓰레드를 사용함에 따라, 동시 접속자 수 증가 시 API 처리용 쓰레드 고갈 위험 문제 발생 가능

🛠️ 해결 방안(Solution)

- 메인 API 기능을 담당하는 쓰레드와 다른 비동기 작업용 보조 쓰레드를 분리하여 특정 기능의 부하가 전체 시스템의 병목이 되지 않도록 격리
  - **'kafkaFallbackExecutor':** 주문 Kafka 메시지 발행 후 예외 처리를 전담하는 별도의 비동기 쓰레드 풀 할당
  - **'notificationExecutor':** SSE 알림 전송 및 지속 연결 관리를 전담하는 별도의 비동기 쓰레드 풀 할당

✅ 결과 및 성과

- API 응답 속도 최적화 및 병목 개선
  - 기존의 Kafka ACK 동기 대기를 제거하고 비동기 처리로 전환하여 HTTP 응답 지연 최소화
- Tomcat Thread 및 DB Connection 안정성 확보
  - 비동기 I/O 작업을 커스텀 쓰레드 풀로 이관하여 동시 부하 상황에서도 Tomcat 쓰레드 풀 고갈 방지하여 DB 커넥션 풀의(pending = 0) 안정성 유지
- 모니터링 가시성 극대화
  - 쓰레드 풀별로 Active, pending 메트릭 수집을 통해, 시스템 부하 발생시 어떤 레이어에서 대기열이 발생하는지 실시간으로 추적,관리 모니터링 환경 구축



💡 문제점(Challenge)

- **주문 건별 db 쓰기 작업 및 락 경합으로 인한 Consumer LAG 증가 및 I/O 부하의 구조적 한계**

🛠️ 해결 방안(Solution)

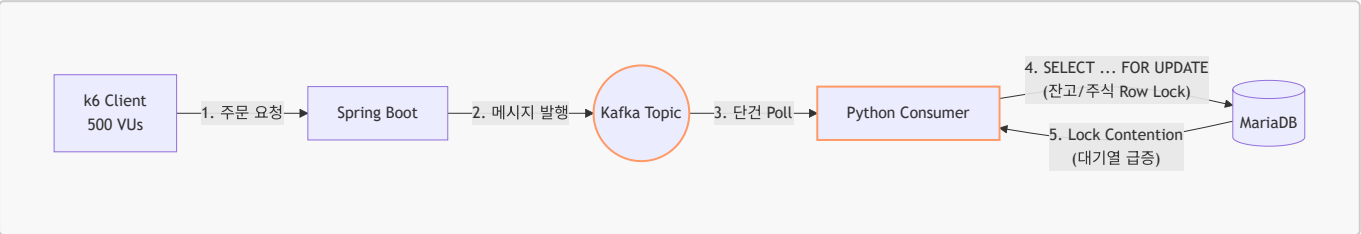
- **Redis 기반 In-Memory 연산 분산 및 배치 Bulk Insert 도입:**

✅ 결과 및 성과

3단계 서비스 아키텍처 흐름 변화

1단계: Python Consumer 내 RDB 비관적 락 (Pessimistic Locking)

Kafka Consumer가 메시지를 순차 수신하여 DB 트랜잭션 내 `SELECT ... FOR UPDATE`로 직접 동시성을 제어함에 따라, DB Lock 대기열 경합 및 Consumer Lag가 극심하던 구조입니다.



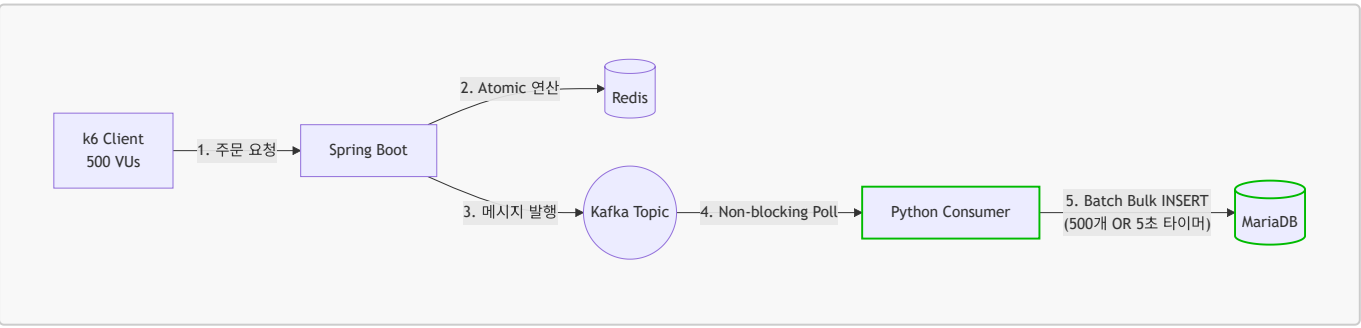
2단계: DB 비관적 락 해제 (Redis 도입 전)

Python Consumer 내 DB Row Lock을 해제하여 메시지 소비 속도와 API 수용량은 증가했으나, 여전히 단건 DB INSERT 처리 한계로 인해 Kafka Lag가 쌓이던 구조입니다.



3단계: Redis 인메모리 체결 & Bulk Insert & Non-blocking Consumer (최종)

Redis Lua Script 기반 인메모리 연산으로 체결 레이어를 이관하고, Kafka Consumer를 Non-blocking 구조(`poll(timeout_ms=1000)`) + 배치 Bulk INSERT로 개편하여 병목을 완전히 해소한 구조



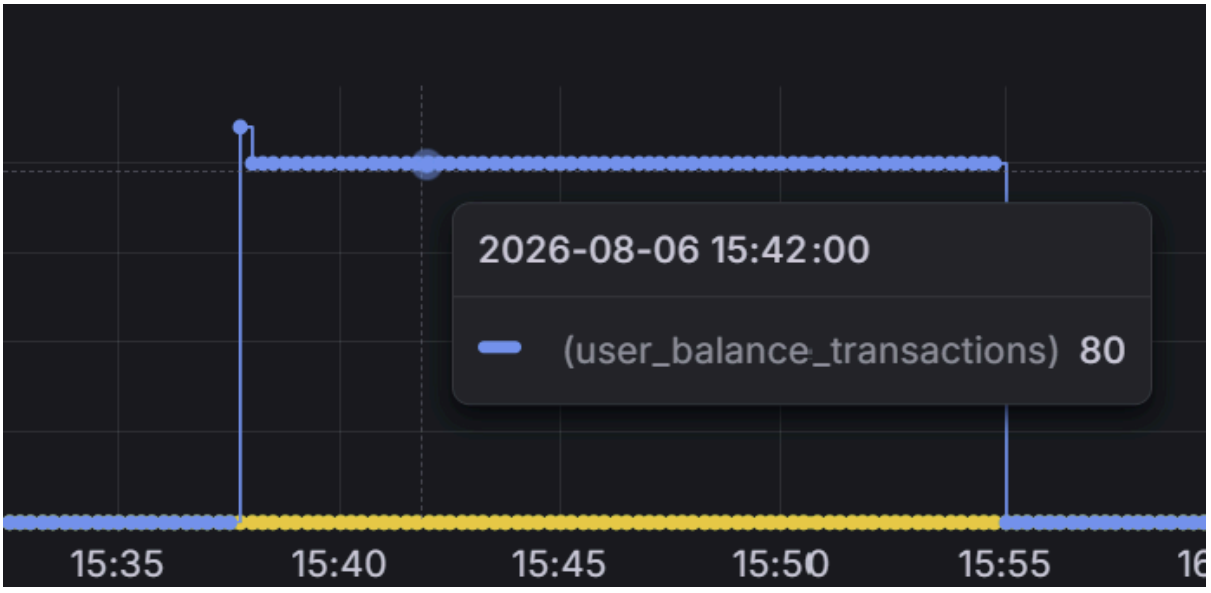
📊 3단계 성능 개선 통합 비교표

평가 지표 (Metrics)	1단계 (Consumer 비관적 락)	2단계 (락 해제)	3단계 (Redis + Bulk Insert)	최종 개선율 (1단계 대비)
초당 처리량 (TPS)	**83.32 req/s**	158.81 req/s	**212.14 req/s**	**+154.6% 향상 (2.54 배)**

평가 지표 (Metrics)	1단계 (Consumer 비관적 락)	2단계 (락 해제)	3단계 (Redis + Bulk Insert)	최종 개선율 (1단계 대비)
총 성공 주문 건수	**22,444 건**	42,697 건	**56,259 건**	**+150.7% 증가**
HTTP 에러율	**0.53%**	0.28%	**0.21%**	**-60.4% 감소**
평균 응답시간 (Avg Latency)	**3.68 s**	1.91 s	**1.43 s**	**-61.1% 단축**
중앙 응답시간 (Median)	**2.54 s**	1.50 s	**1.00 s**	**-60.6% 단축**
P95 응답시간 (P95 Latency)	**6.65 s**	3.42 s	**2.70 s**	**-59.4% 단축**
최대 Consumer Lag (Peak)	**120,000 건**	40,000 건	**10,000 건**	**-91.7% 감소 (1/12 수준)**
Lag 완전 소진 소요 시간	**120 분 (2시간)**	30 분	**5 분**	**-95.8% 단축 (24배 빠른 소모)**

💡 문제점(Challenge)

- **메시지 유입 중단 시 배치 buffer가 flush를 수행하지 않아 데이터가 정체 되는 문제 발생**
  - 기존의 db 작업을 수행하는 로직이 **메시지 수신 이벤트**에 의존하여 flush 여부를 검사했기 때문에, 후속 메시지가 들어올 때까지 DB 반영이 지연 되어 LAG 문제 발생
  - LAG에서 80개의 메시지가 계속 남아있는 병목 발생



  - 테스트 중 강제로 1건의 입금을 발생시키자 버퍼에 남아있던 80건이 한 번에 DB로 처리되는 것을 통해 메모리 정체 원인 최종 확인

```
ubuntu@arm-instance:~/cluster$ kubectl logs deployments/balance-process --tail=100
[Main] 입출금 시스템 가동 및 인프라 초기화 시스템 시동 ...
Connecting to MariaDB...
Successfully connected to MariaDB!
Connecting to Kafka broker at broker:9092...
Successfully connected to Kafka broker
☑ DB Bulk INSERT 성공 : 총 1건 적재 완료
☑ DB Bulk INSERT 성공 : 총 1건 적재 완료
☑ DB Bulk INSERT 성공 : 총 186건 적재 완료
☑ DB Bulk INSERT 성공 : 총 233건 적재 완료
☑ DB Bulk INSERT 성공 : 총 81건 적재 완료
```

🛠️ 해결 방안(Solution)

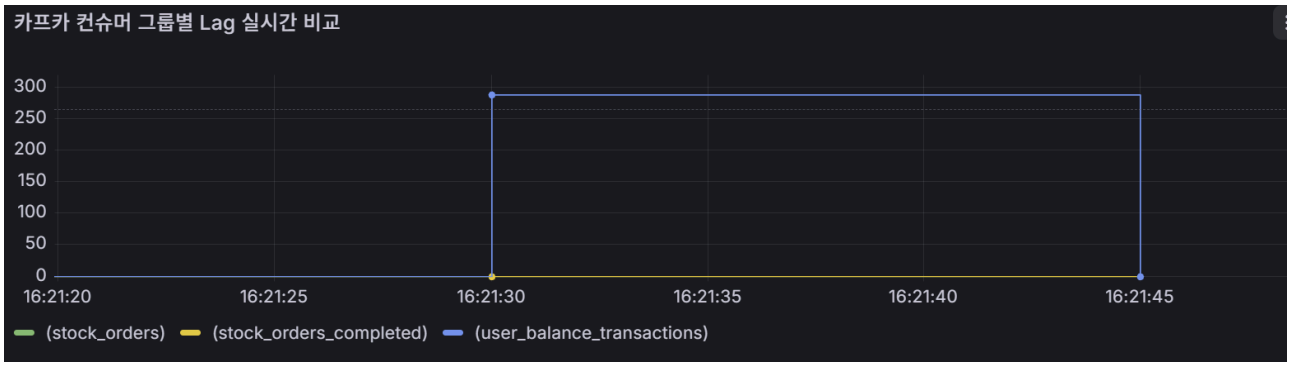
- 타임아웃 기반 non-blocking 폴링 구조로 변경
  - 1초 동안 신규 주문 메시지가 없어도 타임아웃 후에 버퍼 플러시 조건 검사하도록 수정

```
records_dict = consumer.poll(timeout_ms=1000)

if order_buffer and is_flush_required(len(order_buffer), last_flush_time):
  process_flush()
```

✅ 결과 및 성과

- 똑같은 500건의 테스트 수행 결과 0.15초 만에 문제없이 모든 작업 처리 완료




```
ubuntu@arm-instance:~/kafka-set/consumer_balance$ kubectl logs -f deployments/balance-process
[Main] 입출금 시스템 가동 및 인프라 초기화 시스템 시동 ...
Connecting to MariaDB...
Successfully connected to MariaDB!
Connecting to Kafka broker at broker:9092...
Successfully connected to Kafka broker
[2026-08-06 16:21:21] ☑ DB Bulk INSERT 성공 : 총 1건 적재 완료 (소요시간 : 0.0164초 / 16.42ms)
[2026-08-06 16:21:26] ☑ DB Bulk INSERT 성공 : 총 211건 적재 완료 (소요시간 : 0.0775초 / 77.46ms)
[2026-08-06 16:21:31] ☑ DB Bulk INSERT 성공 : 총 288건 적재 완료 (소요시간 : 0.0149초 / 14.86ms)
```

- 단순히 bulk insert를 수행하기 위한 시간 조건과, 크기 조건만 설정해 놓았었는데, 특정 이벤트에 묶여 Lock이 발생하지 않도록 전체 제어 흐름을 고려해야 함을 이해했음